

(c) The function `Dequeue()`:

- returns the next item in the queue, if the queue is **not** empty
- returns `-1` if the queue is empty
- updates the pointers where appropriate.

Write program code for `Dequeue()`

Save your program.

Copy and paste the program code into part **1(c)** in the evidence document.

[5]

(d) The text file `QueueData.txt` stores positive integers. Each integer is on a new line in the file.

The procedure `CreateQueue()`:

- opens the file `QueueData.txt`
- reads in each line from the text file and uses `Enqueue()` to insert each line into the queue
- outputs `"Queue full"` if any item cannot be inserted into the queue
- uses exception handling when opening and reading from the text file.

The procedure needs to work for a file that contains an unknown number of lines.

Write program code for `CreateQueue()`

Save your program.

Copy and paste the program code into part **1(d)** in the evidence document.

[6]

(e) (i) The main program needs extending to call `CreateQueue()`. The main program then adds together all of the integers stored in the queue, using `Dequeue()` to access each integer. The total is then output.

Write program code to extend the main program.

Save your program.

Copy and paste the program code into part **1(e)(i)** in the evidence document.

[5]

(ii) Test your program.

Take a screenshot of the output(s).

Save your program.

Copy and paste the screenshot into part **1(e)(ii)** in the evidence document.

[1]